

RDBMS

UNIT V NOTES

Introduction to PL/SQL

PL/SQL (Procedural Language/SQL) is Oracle Corporation's procedural extension of **SQL** used in the **Oracle Database**.

It allows developers to write **programs that include SQL statements, variables, loops, conditions, and error handling**.

In simple terms:

SQL → used to query and manipulate data

PL/SQL → used to write complete programs that control how SQL runs

Structure of a PL/SQL Block:

A PL/SQL program is written in **blocks**.

Basic structure:

```
DECLARE
    -- Variable declarations
BEGIN
    -- Executable statements
EXCEPTION
    -- Error handling
END;
/
```

Sections of a Block:

Section	Description
DECLARE	Declares variables, constants, cursors
BEGIN	Contains executable statements
EXCEPTION	Handles runtime errors
END	Marks the end of block

Note:

- **DECLARE** and **EXCEPTION** are optional
- **BEGIN** and **END** are mandatory

Example of Simple PL/SQL Program:

```
DECLARE
    message VARCHAR2(50);
BEGIN
    message := 'Hello PL/SQL';
    DBMS_OUTPUT.PUT_LINE(message);
END;
/
```

Output:

Hello PL/SQL

Explanation:

- Variable **message** declared
 - Value assigned using **:=**
 - Output displayed using **DBMS_OUTPUT.PUT_LINE**
-

Variable:

A variable in PL/SQL is a named memory location used to store data temporarily during program execution. The value stored in a variable can change while the program runs.

Variables are declared in the DECLARE section of a PL/SQL block.

Syntax:

DECLARE

 variable_name datatype [:= initial_value];

BEGIN

 -- executable statements

END;

/

variable_name → name of the variable

datatype → type of data stored (NUMBER, VARCHAR2, DATE, etc.)

:= → assignment operator

initial_value → optional starting value

Example of Variable Declaration:

DECLARE

 emp_name VARCHAR2(30);

 salary NUMBER;

BEGIN

 emp_name := 'Ramesh';

 salary := 20000;

 DBMS_OUTPUT.PUT_LINE(emp_name);

 DBMS_OUTPUT.PUT_LINE(salary);

END;

/

Output

Ramesh
20000

Types of Variables in PL/SQL:

1. Scalar Variables

Store single values.

Example:

```
DECLARE

    age NUMBER := 25;

    name VARCHAR2(20) := 'Arun';

BEGIN

    DBMS_OUTPUT.PUT_LINE(name);

END;

/
```

2. Reference Variables

Store values referencing a table column datatype using %TYPE.

Example:

```
DECLARE

    emp_salary employees.salary%TYPE;

BEGIN

    emp_salary := 30000;

END;

/
```

Benefit: If the table datatype changes, the variable automatically adapts.

3. Record Variables

Store multiple related fields together using %ROWTYPE.

Example:

```
DECLARE

    emp_record employees%ROWTYPE;

BEGIN

    SELECT * INTO emp_record

    FROM employees

    WHERE employee_id = 101;

END;
```

/

Constants:

A constant in PL/SQL is a fixed value that cannot be changed during the execution of a program.

Once a constant is declared and assigned a value, it remains the same throughout the program.

Constants are declared in the DECLARE section of a PL/SQL block.

Syntax

DECLARE

 constant_name CONSTANT datatype := value;

BEGIN

 -- executable statements

END;

/

Explanation

constant_name → Name of the constant

CONSTANT → Keyword used to declare a constant

datatype → Type of data (NUMBER, VARCHAR2, DATE, etc.)

:= value → Assigns a fixed value to the constant

Example Program

DECLARE

 pi CONSTANT NUMBER := 3.14;

 radius NUMBER := 5;

 area NUMBER;

BEGIN

 area := pi * radius * radius;

 DBMS_OUTPUT.PUT_LINE('Area of Circle: ' || area);

END;

/

Output

Area of Circle: 78.5

Explanation:

pi is declared as a constant.

Its value cannot be changed during program execution.

Characteristics of Constants:

Value must be assigned at the time of declaration.

Cannot be modified later in the program.

Declared using the CONSTANT keyword.

Helps make programs more readable and secure.

Conditional Statements

Conditional statements in PL/SQL are used to control the flow of execution of a program based on certain conditions.

They allow the program to make decisions and execute specific blocks of code when conditions are true or false.

Types of Conditional Statements in PL/SQL

IF Statement

IF–THEN–ELSE Statement

IF–THEN–ELSIF Statement

CASE Statement

1. IF Statement

Syntax

IF condition THEN

 statement;

END IF;

Example

DECLARE

 num NUMBER := 10;

BEGIN

 IF num > 5 THEN

 DBMS_OUTPUT.PUT_LINE('Number is greater than 5');

 END IF;

END;

/

Explanation:

If the condition is true, the statement inside the IF block will execute.

2. IF–THEN–ELSE Statement

This statement is used when there are two possible conditions.

Syntax

IF condition THEN

statement1;

ELSE

statement2;

END IF;

Example

DECLARE

num NUMBER := 4;

BEGIN

IF num MOD 2 = 0 THEN

DBMS_OUTPUT.PUT_LINE('Even Number');

ELSE

DBMS_OUTPUT.PUT_LINE('Odd Number');

END IF;

END;

/

Explanation:

If condition is true, first statement runs.

If false, ELSE part runs.

3. IF–THEN–ELSIF Statement

Used when there are multiple conditions.

Syntax

IF condition1 THEN

statement1;

ELSIF condition2 THEN

statement2;

ELSIF condition3 THEN

statement3;

ELSE

statement4;

END IF;

Example

DECLARE

marks NUMBER := 75;

BEGIN

IF marks >= 90 THEN

DBMS_OUTPUT.PUT_LINE('Grade A');

ELSIF marks >= 70 THEN

DBMS_OUTPUT.PUT_LINE('Grade B');

ELSIF marks >= 50 THEN

DBMS_OUTPUT.PUT_LINE('Grade C');

ELSE

DBMS_OUTPUT.PUT_LINE('Fail');

END IF;

END;

/

4. CASE Statement

CASE statement is used as an alternative to multiple IF statements.

Syntax

CASE variable

WHEN value1 THEN statement1;

WHEN value2 THEN statement2;

ELSE statement3;

END CASE;

Example

DECLARE

grade CHAR := 'A';

BEGIN

CASE grade

WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');

WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Very Good');

WHEN 'C' THEN DBMS_OUTPUT.PUT_LINE('Good');

ELSE DBMS_OUTPUT.PUT_LINE('Needs Improvement');

END CASE;

END;

/

CURSOR:

A Cursor in PL/SQL is a pointer to a memory area that stores the result of a SQL query.

It allows PL/SQL to process query results row by row.

When a SQL statement returns multiple rows, a cursor helps to retrieve and manipulate each row individually.

Types of Cursors in PL/SQL:

Implicit Cursor

Explicit Cursor

Implicit Cursor

- Automatically created by **Oracle** for SQL statements like INSERT, UPDATE, DELETE, and SELECT INTO.
- No need for programmer to declare it.

Example

DECLARE

emp_name VARCHAR2(30);

BEGIN

SELECT name INTO emp_name

FROM employees

WHERE emp_id = 101;

DBMS_OUTPUT.PUT LINE(emp_name);

END;

/

Explicit Cursor

- Declared by the **programmer**.
- Used when a query returns **multiple rows**.
- Allows processing each row one by one.

Steps for Using Explicit Cursor

1. **Declare the cursor**
2. **Open the cursor**
3. **Fetch data**
4. **Close the cursor**

Syntax

DECLARE

CURSOR cursor_name IS

SELECT column_name FROM table_name;

BEGIN

OPEN cursor_name;

FETCH cursor_name INTO variable;

CLOSE cursor_name;

END;

/

Example Program

DECLARE

CURSOR emp_cursor IS

SELECT name FROM employees;

emp_name employees.name%TYPE;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor INTO emp_name;

EXIT WHEN emp_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(emp_name);

END LOOP;

CLOSE emp_cursor;

END;

/

This program displays **employee names one by one**.

3. Cursor Attributes

Cursor attributes provide information about cursor execution.

Attribute	Description
%FOUND	TRUE if a row is fetched
%NOTFOUND	TRUE if no row is found
%ROWCOUNT	Number of rows fetched
%ISOPEN	Checks whether cursor is open

Example:

EXIT WHEN emp_cursor%NOTFOUND;

PL/SQL Procedures:

A Procedure in PL/SQL is a named block of code that performs a specific task.

It is stored in the Oracle database and can be executed multiple times whenever required.

Procedures help in modular programming, meaning large programs can be divided into smaller manageable parts.

Syntax of PL/SQL Procedure

```
CREATE OR REPLACE PROCEDURE procedure_name
```

```
IS
```

```
BEGIN
```

```
    -- executable statements
```

```
END;
```

```
/
```

Explanation:

CREATE OR REPLACE → Creates a new procedure or replaces an existing one

PROCEDURE procedure_name → Name of the procedure

BEGIN → Start of executable statements

END → End of procedure

Example of Simple Procedure

```
CREATE OR REPLACE PROCEDURE greet_user
```

```
IS
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL');
```

```
END;
```

```
/
```

Executing the Procedure

```
BEGIN
```

```
    greet_user;
```

```
END;
```

```
/
```

Output

```
Welcome to PL/SQL
```

PL/SQL Functions:

A **PL/SQL Function** is a **stored program** in the Oracle Database that **returns a single value** after performing some computation or processing.

- It is similar to functions in programming languages like **C, C++, or Java**.
- A function **must return a value** using the RETURN statement.
- Functions can accept **parameters (inputs)** and process them

Syntax of PL/SQL Function:

```
CREATE OR REPLACE FUNCTION function_name
```

```
(parameter datatype)
```

```
RETURN datatype
```

```
IS
```

```
    variable datatype;
```

```
BEGIN
```

```
    -- executable statements
```

```
    RETURN value;
```

```
END;
```

```
/
```

Explanation

Component	Description
CREATE OR REPLACE	Creates a new function or replaces an existing one
FUNCTION function_name	Name of the function
parameter datatype	Input parameter
RETURN datatype	Specifies the return data type
IS / AS	Declares variables
BEGIN	Start of executable section
RETURN	Returns the result
END	End of function

Example of a Simple Function

Example 1: Function to Add Two Numbers

```
CREATE OR REPLACE FUNCTION add_numbers
```

```
(a NUMBER, b NUMBER)
```

```
RETURN NUMBER
```

```
IS
```

```
    result NUMBER;
```

```
BEGIN
```

```
    result := a + b;
```

```
    RETURN result;
```

```
END;
```

```
/
```

Execution

```
SELECT add_numbers(10,20) FROM dual;
```

Output

30

PL/SQL Exception Handling:

Exception Handling in **PL/SQL** is a mechanism used to **handle runtime errors** that occur during program execution.

- An **exception** is an **error or unexpected event** that interrupts the normal flow of a program.
- PL/SQL provides a way to **detect errors and handle them gracefully** instead of terminating the program.

✦ Example Errors

- Dividing a number by zero
- Fetching data that does not exist
- Retrieving multiple rows when only one row is expected

Structure of Exception Handling

The Exception Handling section is written at the end of a PL/SQL block.

Syntax

```
DECLARE
```

```
-- variable declaration
```

```
BEGIN
```

```
-- executable statements
```

```
EXCEPTION
```

```
WHEN exception_name THEN
```

```
-- error handling statements
```

```
END;
```

```
/
```

Components

SectionDescription

DECLARE Defines variables

BEGIN Contains executable statements

EXCEPTION Handles runtime errors

END End of block

Types of Exceptions in PL/SQL

There are three types of exceptions.

1. Predefined Exceptions

These are built-in exceptions provided by PL/SQL.

Exception	Description
-----------	-------------

NO_DATA_FOUND	No rows returned by SELECT
---------------	----------------------------

TOO_MANY_ROWS	Query returns more than one row
---------------	---------------------------------

ZERO_DIVIDE	Division by zero
-------------	------------------

INVALID_NUMBER	Invalid number conversion
----------------	---------------------------

2. Non-Predefined Exceptions

These are **Oracle errors that do not have predefined names**.

They can be handled using:

- PRAGMA EXCEPTION_INIT

3. User-Defined Exceptions

These exceptions are **created by the programmer**.

Steps:

1. Declare the exception
2. Raise the exception using RAISE
3. Handle it in the exception block

4. Using WHEN OTHERS

WHEN OTHERS handles all other exceptions that are not explicitly handled.

PL/SQL Triggers:

A Trigger in PL/SQL is a stored program that automatically executes when a specific event occurs in a database table.

Triggers are mainly used in Oracle Database to maintain data integrity, enforce business rules, and audit database changes.

📌 Example:

When a new employee record is inserted into a table, a trigger can automatically store the details in an audit table.

Common Trigger Events

Event	Description
-------	-------------

INSERT	Trigger fires when new data is inserted
--------	---

UPDATE	Trigger fires when data is updated
--------	------------------------------------

DELETE	Trigger fires when data is deleted
--------	------------------------------------

Syntax of a Trigger

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE | AFTER INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
DECLARE
    -- variable declarations
BEGIN
    -- trigger body
END;
/
```

Explanation

Keyword	Meaning
CREATE TRIGGER	Creates the trigger
BEFORE / AFTER	Time when the trigger executes
INSERT / UPDATE / DELETE	Triggering event
ON table_name	Table on which trigger is created
FOR EACH ROW	Executes for every row affected

Types of Triggers

1. BEFORE Trigger

Executed before the DML operation occurs.

2. AFTER Trigger

Executed **after the DML operation** is completed.